SIMATS ENGINEERING

ASSESSMENT TOOL 2 — DETAILED ANSWER SCRIPT

Course Name: Artificial Intelligence Course Code: CSA17

CO2: Search & Game-Solving Techniques (Greedy, A, CSP, Minimax, Alpha-Beta) — BL3*

Scenario Based Assignment

Question 1 — AI Drone Delivery in a Flood-Affected Region

Scenario Recap: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. Roads may suddenly become inaccessible, and weather affects travel cost. The drone has a heuristic estimate (straight-line distance), partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather. Due to urgency, the drone must decide routes quickly while minimizing delay and risk.

This scenario is a classic informed-search problem layered with real-time uncertainty. Below, each requested task is addressed in depth, followed by a worked comparison and a final justified recommendation.

i. Behaviour of Greedy Best-First Search (GBFS) Under Uncertainty

Greedy Best-First Search selects, at every step, the node n that minimizes the heuristic function h(n) — the estimated straight-line distance from that node to the destination. Critically, it never looks at g(n), the actual cost already paid to reach n. In the drone-delivery context, this means the algorithm behaves like a myopic navigator: at every junction it asks only "which of my next reachable points looks geometrically closest to the hospital or drop point?" and moves there, without asking "how much fuel, time, or risk has this route already cost me, or is likely to cost me next?"

Concretely, imagine the drone is choosing between two intermediate waypoints, A and B. Waypoint A is 2 km closer to the destination in straight-line terms but lies across a temporarily flooded, high-turbulence corridor; waypoint B is 3 km further away in straight-line terms but is a safe, currently open corridor. GBFS will unconditionally prefer A because it never evaluates the real traversal cost or risk of the corridor — it only compares straight-line distances.

Strengths of GBFS in this context

- Speed: because it expands the single most heuristic-promising node at each step and largely ignores cost bookkeeping, GBFS typically explores far fewer nodes than A* or uninformed search, producing a route almost instantly — valuable when a decision must be made in seconds during an emergency.
- Low memory footprint: it does not need to retain and continuously re-rank costs across a large frontier, so it is well suited to a resource-constrained onboard flight computer.
- Effective when the heuristic is highly reliable and terrain is largely static: if the straight-line-distance heuristic correlates strongly with real traversal cost (e.g., open plains with no obstacles), GBFS's chosen path is often close to the true best route despite ignoring cost.

Weaknesses of GBFS in this context

- Sub-optimality: GBFS provides no guarantee of finding the least-cost (fastest/safest) path — it can be lured toward a route that looks close on a map but is actually blocked, flooded, or far riskier.
- Incompleteness / looping risk: in a graph with cycles, without careful duplicate-state checking, GBFS can oscillate between nodes that each look attractive relative to each other but do not make real progress, wasting critical time near a flooded obstacle.
- Extreme sensitivity to uncertainty: because it has no memory of accumulated cost, a single wrong early greedy choice is never "paid back" or reconsidered later — a bad decision near the start of the route compounds and cannot self-correct without a full restart.
- Poor risk-awareness: GBFS has no built-in way to weigh danger/urgency trade-offs since its only signal is proximity, not safety or feasibility.

ii. Behaviour of A* Search — Balancing Cost and Heuristic

A* evaluates every candidate node n using the evaluation function f(n) = g(n) + h(n), where g(n) is the exact, real path cost accumulated so far (which factors in terrain difficulty and weather-driven delay) and h(n) is the estimated remaining straight-line distance to the goal. This dual accounting is what allows A* to "balance" the two competing signals: g(n) keeps the search grounded in the real, already-experienced difficulty of the route, while h(n) keeps the search goal-directed and efficient rather than exploring blindly like an uninformed search.

Returning to the earlier example: A* would compute g(A) versus g(B) using actual weather-adjusted terrain cost. Because corridor A is flooded/turbulent, its real edge cost is very high, so even though h(A) is small, f(A) = g(A) + h(A) becomes large. Corridor B, while geometrically longer, has a low real cost, so f(B) ends up smaller. A* therefore correctly prefers B — something GBFS structurally cannot do.

Key properties of A* here

- Completeness and optimality: provided the heuristic is admissible (never overestimates true remaining cost — which straight-line distance naturally satisfies on a 2D/3D map, since no real route can be shorter than a straight line) and consistent, A* is guaranteed to find the optimal path if one exists.
- Adaptivity to blocked paths: when a real-time update reports a corridor is now blocked, the edge cost for that segment can be set to infinity (or removed), and A* — especially in incremental forms such as D*-Lite or Lifelong Planning A* — can efficiently re-derive a new optimal route by reusing much of its previous search effort rather than restarting from scratch.
- Cost/urgency trade-off: because A* is cost-aware, its objective function can be engineered to directly encode "delay and risk" (e.g., g(n) could combine expected travel time and a risk penalty for turbulent/flooded segments), letting the algorithm directly optimize what actually matters operationally, not just geometric proximity.

Costs of using A* here

- A* must maintain an open list (priority queue) and a closed list, expanding more nodes than GBFS in general, which increases computation time and memory usage — a genuine concern for constrained onboard hardware and hard real-time deadlines.
- If the heuristic weakly correlates with real cost (e.g., very complex flooded terrain where straight-line distance is a poor proxy), A* may need to expand a much larger fraction of the graph to guarantee optimality.

iii. Impact of Heuristic Accuracy on Both Algorithms

Heuristic accuracy plays a fundamentally different structural role in each algorithm.

Aspect	Greedy Best-First Search	A* Search
Role of heuristic	The only signal used to decide expansion order	One of two signals (combined with real cost g(n))
Effect of an accurate heuristic	Produces a good route quickly, close to optimal	Produces the optimal route with very high efficiency (few nodes expanded)
Effect of an inaccurate / misleading heuristic	Can produce a badly suboptimal or unsafe route with no way to self-correct	Slower (expands more nodes) but if admissible, still guaranteed optimal
Effect of an overestimating (inadmissible) heuristic	No formal guarantee lost since GBFS never guaranteed optimality anyway	Optimality guarantee can be lost — A* may return a suboptimal path
Overall dependency	Correctness itself depends entirely on heuristic quality	Only efficiency depends on heuristic quality; correctness depends on admissibility

In the flood scenario specifically, a straight-line-distance heuristic is admissible (it can never overestimate real travel distance across terrain, since terrain paths are always at least as long as the straight line), so A* retains its optimality guarantee even when flooding makes the heuristic a poor predictor of real cost — it will simply expand more nodes. GBFS has no such safety net: a poor correlation between straight-line distance and real flood-adjusted cost directly translates into a poor, potentially dangerous route choice.

iv. Effect of Dynamic Changes (Blocked Paths) on Search Performance

- Both algorithms are conventionally designed for static graphs; a newly blocked road invalidates part of any previously computed plan and, in the worst case, requires re-searching from the current position.
- GBFS reacts poorly to dynamic blocking: since it never tracked real cost, when its previously preferred "closest-looking" route becomes blocked, it has no cost memory to fall back on and must essentially restart heuristic-driven exploration, sometimes oscillating between similarly heuristic-scored alternatives that are both now degraded.
- A* supports far more graceful adaptation: incremental replanning algorithms built on A* (D*-Lite, LPA*) update only the portion of the search graph affected by the blockage and reuse the bulk of previous computation, which is critical in a continuously changing flood environment where blockages may appear and clear repeatedly.
- Both algorithms suffer increased computational load under frequent replanning; if updates arrive faster than the drone's onboard processor can re-plan, the system must impose a hard time budget (e.g., anytime search) so that continuous re-planning itself does not become the source of dangerous delay.
- A secondary effect of dynamic blocking is that any pre-computed path becomes a soft plan rather than a fixed commitment — the drone's control software must treat the route as continuously revisable, requiring an architecture where sensing, replanning, and execution are tightly interleaved (an online search loop) rather than a single "plan once, then fly" pipeline.

v. Recommendation and Justification

The most suitable algorithm for this scenario is A* Search, ideally implemented as an incremental / real-time variant such as D*-Lite or Anytime Repairing A* (ARA*), for the following reasons:

- Optimality and safety: because human lives depend on timely and reliable medicine delivery, the drone should not merely appear to be moving toward the goal (as GBFS would ensure) — it should genuinely minimize real delay and risk, which only a cost-aware algorithm like A* can guarantee under an admissible heuristic.
- Real-time feasibility: pure exhaustive A* could be too slow under strict deadlines, but anytime and incremental variants of A* allow the drone to obtain a "good enough" route almost immediately and then refine it if additional computation time is available before the flight must begin, directly addressing the urgency constraint.
- Robustness to dynamic blocking: A*'s incremental replanning variants are specifically designed for exactly this kind of continuously changing graph, making it structurally the better fit compared to GBFS, which has no principled mechanism for cost-based recovery.
- Role for GBFS as a fallback only: GBFS may still have a supporting role — for instance, generating an extremely fast first-guess route to start the drone moving while a more careful A*-based plan is computed in parallel — but it should never be the final authority on route selection given its lack of optimality and poor handling of blocked paths.

In summary, GBFS trades correctness for speed, while A* trades some speed for guaranteed correctness (given an admissible heuristic); given the safety-critical, emergency nature of the flood-relief scenario, the balance clearly favours A* (in an incremental, anytime form) as the primary decision-making algorithm, with GBFS-style heuristics used only for fast provisional estimates.

Question 2 — Smart City Traffic Signal Optimization

Scenario Recap: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections, aiming to minimize total waiting time, fuel consumption, and traffic congestion, while continuously adjusting to dynamically changing traffic flow, an extremely large search space, and the risk of local optima.

Formulating the Problem as an Optimization Problem

- State representation: a complete configuration assigning timing parameters — green duration, red duration, cycle length, and phase offset relative to neighbouring intersections — to every signal in the city.
- Objective (cost) function: a weighted combination such as Cost = w1 × TotalWaitingTime + w2 × FuelConsumption + w3 × CongestionIndex, to be minimized (equivalently, maximized as a negated "efficiency score"), where the weights let planners prioritize, e.g., emissions versus pure throughput.
- Search space: the Cartesian product of all possible timing parameters across all intersections — combinatorially explosive, since intersections are not independent (changing one signal's timing shifts the arrival pattern at downstream intersections).
- Constraints/dynamics: real traffic flow changes continuously (rush hour vs. off-peak, accidents, events), so the "optimal" configuration itself is a moving target, not a single fixed answer to be solved once.

Why traditional (complete/exhaustive) search is inefficient here

- State-space size: with hundreds of intersections, each with multiple continuous or fine-grained discrete timing parameters, the total number of configurations is astronomically large — exhaustive enumeration or classical uninformed search (BFS/DFS) is computationally intractable.
- No natural goal test: unlike a pathfinding problem with a clear "reached the goal" condition, this is a continuous optimization problem without a single terminal state — traditional path-search algorithms such as A* are designed for goal-directed pathfinding, not for this kind of multi-objective continuous parameter tuning.
- Non-stationary environment: traditional complete search methods generally assume a static problem definition; here, the "correct answer" changes minute to minute as traffic patterns shift, so a single offline exhaustive solve would be outdated before it could even be applied.
- Consequently, this problem calls for iterative-improvement / metaheuristic local search methods rather than a complete, exhaustive search algorithm.

i. Hill Climbing in This Scenario and Its Limitations

Hill Climbing starts from some initial timing configuration (e.g., a default or previous best plan) and repeatedly examines neighbouring configurations — such as increasing or decreasing one intersection's green-phase duration by a small increment — moving to any neighbour that improves the overall objective function, and stopping once no neighbour offers improvement.

```
function HILL-CLIMBING(problem):
  current = problem.INITIAL-STATE
  loop:
    neighbor = the highest-valued successor of current
    if VALUE(neighbor) <= VALUE(current):
```

```
        return current   // reached a peak
    current = neighbor
```

- Limitation — gets trapped at local optima: hill climbing has no mechanism to accept a temporarily worse configuration, so once it reaches a configuration that is better than all immediate neighbours (but not the best globally), it stops there permanently.
- Limitation — no backtracking / no memory of alternative branches: if an early greedy improvement leads toward a poor region of the search space, hill climbing cannot revisit and try a different direction.
- Limitation — sensitivity to interacting variables: intersections are coupled (a change at one affects downstream traffic at another), so single-variable-at-a-time improvement moves can be poorly suited to problems that require coordinated multi-variable change, as discussed under "ridges" below.
- Limitation — highly dependent on the starting point: different initial timing configurations can lead hill climbing to entirely different (and possibly very different quality) local optima.

ii. Local Maxima, Plateaus, and Ridges — Real-World Interpretation

Issue	Technical Definition	Real-World Traffic Interpretation
Local maxima	A state better than all its neighbours, but not the global best	A timing plan that nicely optimizes one busy corridor's flow, while a nearby residential zone remains poorly timed; no single small tweak improves the citywide total, even though a very different plan would be far better overall
Plateaus	A flat region where neighbouring states have nearly equal value	Shifting a green-phase by one or two seconds produces no measurable change in waiting time, so the search has no clear "uphill" direction to follow and can wander indefinitely
Ridges	A sequence of local maxima that requires simultaneous multi-variable moves to cross, since single-step moves each look like a decrease	A city-wide "green wave" along a corridor requires several consecutive intersections to shift their offsets together; changing them one at a time (as pure hill climbing does) can make things briefly worse even though the combined coordinated change would substantially help

iii. Simulated Annealing and Genetic Algorithms as Solutions

Simulated Annealing (SA): borrows the physical metaphor of cooling metal. It maintains a "temperature" parameter T that starts high and gradually decreases. At each step, SA considers a random neighbouring configuration; if it is better, it is accepted; if it is worse, it is still accepted with probability roughly e^(-ΔE / T), where ΔE is how much worse the new state is. Early on (high T), SA freely accepts worse moves, allowing it to escape local maxima and traverse plateaus; as T cools, it increasingly behaves like hill climbing, settling into a strong final configuration.

```
function SIMULATED-ANNEALING(problem, schedule):
   current = problem.INITIAL-STATE
   for t = 1 to infinity:
      T = schedule(t)
      if T == 0: return current
```

```
next = a randomly selected neighbor of current
deltaE = VALUE(next) - VALUE(current)
if deltaE > 0: current = next
else: current = next with probability exp(deltaE / T)
```

Genetic Algorithms (GA): maintain a population of candidate timing-plan "chromosomes" (e.g., a vector of green-durations/offsets per intersection). Each generation: (1) fitness is evaluated using the same waiting-time/fuel/congestion objective, (2) fitter individuals are more likely to be selected as parents, (3) crossover combines segments of two parent plans (e.g., taking one parent's timing for intersections 1–50 and another's for 51–100), and (4) mutation randomly perturbs some timings to maintain diversity. Over many generations, the population converges toward strong, often near-global-optimal solutions.

- GA directly solves the "ridge" problem described above, because crossover can combine several coordinated intersection changes from a successful parent plan in a single step, rather than requiring the search to discover the coordinated change one variable at a time.
- GA's population-based nature also provides implicit parallel exploration of many regions of the search space simultaneously, substantially reducing (though not eliminating) the risk of premature convergence to one local optimum, especially if diversity-preservation techniques (e.g., mutation rate tuning, niching) are used.
- Both SA and GA are anytime-style algorithms: they can be stopped early (e.g., before the next signal-timing update cycle) and still return a usable, improved solution, which fits the continuously changing operational requirement of a live traffic system.

iv. Recommendation Based on Scalability and Adaptability

Criterion	Hill Climbing	Simulated Annealing	Genetic Algorithm
Escapes local optima	No	Yes (probabilistically)	Yes (population diversity)
Handles coordinated/ridge-type moves	Poor	Moderate	Strong (via crossover)
Scalability to hundreds of intersections	Simple but weak solution quality	Good, single-solution refinement	Good, naturally parallelizable across population
Adaptability to continuously changing traffic	Fast but low quality	Can be restarted / reheated periodically	Can evolve continuously with fresh data injected each generation
Computational cost per iteration	Low	Low–Moderate	Higher (maintains a population)

The recommended overall approach is a Genetic Algorithm as the primary optimizer for periodic, citywide re-optimization of baseline signal-timing plans — optionally hybridized with local search or simulated annealing for fine-tuning individual intersections between major GA runs (a memetic algorithm). This is justified because:

- Scalability: GA's population-based, naturally parallelizable structure scales more gracefully across hundreds of interacting intersections than a single-point search such as hill climbing or even SA.
- Adaptability: new real-time traffic data can be periodically re-injected into the GA's fitness evaluation, letting the population continue evolving in response to changing conditions rather than freezing at a one-time static optimum.

- Practical layering: fast, cheap local hill-climbing-style micro-adjustments can still be used for minor, real-time tuning between major GA/SA re-optimization cycles, balancing computational cost against solution quality — giving the system both a strong long-run plan and rapid short-term responsiveness.

Question 3 — Autonomous Mars Rover Exploration

Scenario Recap: An autonomous rover explores unknown Martian terrain, must navigate safely to collect samples, avoid hazards such as craters and rocks, operate with limited sensing capability, and make decisions without a complete map — updating its knowledge as it explores, making this a real-time decision-making problem.

i. Why This Problem Requires an Online Search Agent Instead of Offline Search

Offline search algorithms assume the agent has complete prior knowledge of the state space (the map, all obstacles, all costs) before it begins acting; the entire solution path is computed in advance and then executed "blindly," without further sensing feedback during execution. This assumption is fundamentally violated on Mars: the terrain map is not known ahead of time, sensing range is limited, and hazards such as craters and loose rocks are only discovered as the rover physically approaches them.

- An online search agent instead interleaves computation with actuation: it takes a single action, observes the actual resulting state through its sensors, updates its internal knowledge of the environment, and only then computes/selects its next action — repeating this sense–plan–act loop continuously.
- This tight loop allows the rover to react immediately to hazards or terrain features the instant they become observable, rather than being committed to a fixed pre-planned route that the very first step might already invalidate.
- Formally, online search is the correct model whenever the agent has no advance model of the successor function/cost structure of unexplored regions — exactly the situation of a planetary rover in unmapped terrain.

ii. Challenges of Partial Observability and Dynamic Environments

- Partial observability: the rover's sensors (cameras, LIDAR, etc.) only reveal a bounded local radius around its current position. It cannot be certain what terrain lies beyond that radius, which can lead to costly detours, dead-end backtracking, or highly conservative movement once hidden hazards are discovered late — meaning worst-case online-search performance can be significantly worse than a hypothetical offline-optimal plan computed with full map knowledge (a gap formally captured by the notion of "competitive ratio").
- Irreversibility of some actions: certain moves — such as driving over the lip of a crater or onto unstable regolith — may be effectively irreversible or catastrophic. This means the rover cannot rely purely on unrestricted trial-and-error exploration the way an online agent could in a fully safe/reversible environment; it must incorporate conservative, risk-averse action selection near uncertain terrain.
- Non-stationary/dynamic terrain: dust storms, wind-driven sand movement, or extreme temperature-driven surface changes mean even previously mapped, already-traversed areas may need periodic re-verification rather than being trusted as permanently known.
- Severe communication delay: the round-trip signal delay between Mars and Earth (several minutes to tens of minutes depending on orbital position) makes real-time human tele-operation infeasible for moment-to-moment hazard avoidance, which greatly increases the importance — and the responsibility — placed on the rover's autonomous online decision-making capability.

- Resource constraints: limited onboard computation, power, and time-of-day (solar-powered) operating windows further constrain how much online search/replanning the rover can practically perform per decision cycle.

iii. How the Rover Builds and Updates Its Internal Model

- Sensor fusion: as the rover moves, onboard cameras, LIDAR/stereo vision, and other sensors capture local terrain data (elevation, texture, obstacles), which is fused into an incrementally constructed representation — commonly an occupancy grid map or a topological graph of traversable versus blocked regions.
- Incremental updating: newly discovered obstacles (craters, large rocks) are marked as hazards in the model; conversely, cells previously assumed traversable that prove impassable upon closer approach are updated and subsequently avoided in future planning.
- Belief-state maintenance: because sensing is imperfect and partial, the rover typically maintains some notion of confidence/uncertainty per grid cell (e.g., "unknown," "likely traversable," "confirmed blocked") rather than a simple binary map, allowing it to make risk-informed decisions about whether to explore an uncertain region or take a known-safe detour.
- Local re-planning: each time meaningful new information arrives, the rover essentially re-runs a search (e.g., an incremental variant of A* such as D*-Lite) over the currently known portion of the map, efficiently reusing as much of the previous search computation as possible rather than replanning from scratch every cycle.

iv. Online Search vs Uninformed and Informed Offline Search

Property	Uninformed Search (e.g., BFS/DFS)	Informed Offline Search (e.g., A*)	Online Search (e.g., LRTA*, D*-Lite)
Requires map in advance?	Yes — fully known graph assumed	Yes — fully known graph and heuristic assumed	No — explores and builds the map as it acts
Uses a heuristic?	No	Yes, guides search toward goal	Yes, typically combined with local re-planning
Can react to new hazards mid-execution?	No	No (would require a full offline re-solve)	Yes — natively designed for this
Optimality guarantee	Complete but not cost-optimal in general (unless uniform-cost)	Optimal given an admissible heuristic, but only for the *assumed known* map	Optimal only relative to information known so far; may be suboptimal compared to a hypothetical fully-informed offline solution
Suitability for Mars rover	Infeasible — no advance map exists	Infeasible — no advance map exists	The only feasible option given the problem's true information constraints

In short, uninformed and informed offline search are both elegant tools for problems where the full state space is known beforehand, but neither can be directly applied here because the very premise — full advance knowledge — is false for planetary exploration. Online search agents combine the goal-directed efficiency ideas of informed search with the ability to act safely under incomplete knowledge, at the acknowledged cost of potentially extra exploration or backtracking relative to a theoretical, unattainable offline-optimal plan.

v. Justification of the Most Suitable Approach

An online, informed search agent — for example, an online variant of A* such as D*-Lite, or Learning Real-Time A* (LRTA*), combined with conservative, risk-aware action selection — is the most suitable approach for this scenario.

- Handling uncertainty: it makes decisions based only on currently available information and continuously updates as more of the environment is sensed, which is the only workable strategy when no advance map exists.
- Adaptability and efficiency: incremental replanning reuses prior search computation instead of restarting from scratch every time new hazards are discovered, which is essential given the rover's limited onboard computing power and power budget.
- Safety: combining online search with cautious action selection near unknown or potentially irreversible-risk regions (e.g., preferring to skirt the edge of an unexplored area rather than cut directly across it) balances exploration efficiency against the rover's physical safety — which is the paramount priority for a costly, irreplaceable space asset operating many light-minutes from any possible rescue or repair.
- This combination — online, informed, incremental search plus conservative risk-aware action policies — is the standard and well-justified real-world approach used by actual planetary rovers (e.g., NASA's Mars rovers use onboard autonomous navigation systems built on these same underlying principles).

Question 4 — University Exam Timetabling as a CSP

Scenario Recap: A university needs an AI system to automatically generate exam timetables subject to several constraints: no student has overlapping exams, a limited number of exam halls, certain subjects must be scheduled before others, faculty availability constraints, and special accommodations for some students. Complexity increases sharply as the number of courses and students grows.

Formulating the Problem as a Constraint Satisfaction Problem (CSP)

A CSP is formally defined by a triple (Variables, Domains, Constraints). Mapping the timetabling scenario onto this triple:

i. Variables

- One variable per exam/course to be scheduled: X1, X2, ..., Xn, where each Xi represents "the assigned (time-slot, hall) pair for course i."
- Optionally, this can be split into two coupled variable sets — a time-slot variable Ti and a hall variable Hi per course — which can simplify expressing hall-capacity and faculty-availability constraints separately from scheduling-order constraints.

Domains

- For each variable Xi, the domain D(Xi) is the set of all valid (time-slot, hall) combinations available during the examination period — e.g., {Day1-Slot1-HallA, Day1-Slot1-HallB, Day1-Slot2-HallA, Day1-Slot2-HallB, ..., DayN-SlotM-HallK}.
- Domain size can be pruned in advance using obvious restrictions, e.g., a hall too small for a course's enrolled student count is simply excluded from that course's domain from the outset, reducing the effective search space before search even begins.

Constraints

Constraint Type	Formal Description	Real-World Meaning
No overlapping exams for a student	Binary constraint: for any two courses i, j sharing at least one enrolled student, Ti != Tj	A student is never scheduled for two exams at the same time
Hall capacity	For each (time-slot, hall) pair, sum of enrolled students of all courses assigned there <= hall capacity	No hall is over-booked beyond its seating limit
Precedence between subjects	If subject A must precede subject B, then Ti(A) < Ti(B) in the timetable ordering	Prerequisite/sequenced subjects are examined in the required order
Faculty availability	Course i requiring faculty member F cannot be assigned a time-slot in F's unavailable set	An exam is never scheduled when its required invigilator/faculty is unavailable
Special accommodations	Unary/binary constraints, e.g., student needs a specific hall-type or extra time buffer between consecutive exams	Accessibility and fairness requirements are respected for students who need them

This constraint set includes both hard constraints (must never be violated, e.g., no student double-booked) and potentially soft constraints (preferably satisfied, e.g., minimizing back-to-back exams for students) — in practice,

a real deployment often treats fairness/preference constraints as soft, optimized after all hard constraints are satisfied.

ii. How Backtracking Search Works in This Scenario

Backtracking search assigns values to variables one at a time — for example, course by course — checking relevant constraints incrementally after each assignment, and backtracks (undoes the most recent assignment and tries an alternative value) whenever the partial assignment is found to violate a constraint.

```
function BACKTRACKING-SEARCH(csp):
  return BACKTRACK({}, csp)

function BACKTRACK(assignment, csp):
  if assignment is complete: return assignment
  var = SELECT-UNASSIGNED-VARIABLE(csp)
  for value in ORDER-DOMAIN-VALUES(var, assignment, csp):
    if value is consistent with assignment:
      add {var = value} to assignment
      result = BACKTRACK(assignment, csp)
      if result != failure: return result
      remove {var = value} from assignment
  return failure
```

- Worked example: after assigning Course A to Day1-Slot1-HallA, the algorithm proceeds to Course B. If B shares students with A, the no-overlap constraint forbids B from also being placed at Day1-Slot1; if the current candidate value for B violates this, backtracking search simply tries B's next domain value (a different slot or hall) instead.
- If no valid value exists for B given A's current assignment, the search backtracks further — undoing A's assignment and trying a different (slot, hall) pair for A — continuing until either a fully consistent timetable is found or the domain is exhausted (indicating, at that branch, no solution is possible).
- Heuristics dramatically improve raw backtracking performance in this large space: the Minimum Remaining Values (MRV) heuristic selects next the course with the fewest legal remaining values (e.g., a very large course with many student overlaps and few compatible halls), attacking the hardest decisions first so failures are discovered earlier; the Degree Heuristic breaks ties by choosing the course involved in the most constraints with other yet-unassigned courses, since resolving highly-connected variables early tends to prune the remaining search space more effectively.
- Value-ordering heuristics such as Least Constraining Value can also help — preferring, for the current course, the (slot, hall) choice that rules out the fewest options for other, not-yet-assigned courses, keeping the search more flexible for later decisions.

iii. How Constraint Propagation (Forward Checking) Improves Efficiency

Forward checking is invoked immediately after each variable assignment: for every not-yet-assigned variable that shares a constraint with the just-assigned variable, forward checking removes from that variable's domain any value now known to be inconsistent.

- Example: once Course A is fixed to Day1-Slot1-HallA, forward checking immediately removes Day1-Slot1 (in any hall) from the domain of every other course that shares at least one student with A, since assigning any of them to that slot would now violate the no-overlap constraint.

- Early failure detection: if forward checking ever reduces some variable's domain to empty, the search can immediately declare failure and backtrack, without having to first assign further variables and only discover the conflict several steps later — this can save enormous amounts of wasted search effort in a problem of this scale.
- Stronger propagation — arc-consistency (AC-3): goes further than simple forward checking by ensuring that for every remaining value of one variable, there exists at least one compatible value in every constraint-connected neighbour's domain, not merely the just-assigned variable's neighbours. This can catch some conflicts that forward checking alone (which only looks one step ahead, from the newly assigned variable) would miss until later in the search.
- Practically, combining MRV/degree-heuristic variable ordering with forward checking (or full AC-3) is the standard, well-justified strategy for making CSP-based timetabling tractable at university scale, since it prunes the enormous domain space early and aggressively rather than relying on brute-force backtracking alone.

iv. Real-World Challenges and Best Strategy for Scalability

- Dense constraint graphs: as course and student counts grow, the number of course pairs sharing at least one student grows rapidly, producing a very densely connected constraint graph that is intrinsically harder for backtracking search to satisfy efficiently.
- Resource scarcity: limited exam halls and limited faculty availability windows create tight bottleneck constraints, especially during peak exam periods when many courses compete for the same in-demand slots.
- Competing soft objectives: beyond the hard constraints, universities typically also want to minimize consecutive exams for the same student, balance load across days, and respect preferences — objectives that pure CSP satisfaction does not directly optimize for, requiring either a weighted soft-constraint CSP formulation or a secondary optimization pass after a feasible timetable is found.
- Recommended scalable strategy: combine backtracking search with MRV + degree-heuristic variable ordering and forward checking / AC-3 constraint propagation to prune the search space aggressively; for very large instances where an exact solution may be slow, supplement with local-search repair methods such as min-conflicts (start from a complete but possibly inconsistent assignment and iteratively repair the most-violated variables) to obtain a good, feasible timetable quickly, and then apply further optimization for soft-preference improvement.
- Problem decomposition also aids scalability in practice: scheduling by department, semester, or programme first, and only later resolving the smaller number of genuine cross-department conflicts, can substantially shrink the effective search space compared to solving the entire university's timetable as one monolithic CSP.

Question 5 — Strategic Game AI: Minimax and Alpha-Beta Pruning

Scenario Recap: A gaming company is developing an AI opponent for a real-time strategy (RTS) game. The AI must compete against human players, make decisions within strict time limits, and evaluate complex game states across an extremely large, computationally expensive decision tree.

i. How the Minimax Algorithm Models This Problem

Minimax models the RTS game as a two-player, zero-sum adversarial search tree: the AI is the maximizing player, and the human opponent is treated as the minimizing player, with turns alternating down the tree. Each terminal (or, in practice, depth-limited) node is assigned a numeric value by an evaluation function, and this value is propagated back up the tree: at MAX nodes, the algorithm takes the maximum of children's values (the AI's best available outcome); at MIN nodes, it takes the minimum (assuming the opponent plays to minimize the AI's outcome).

```
function MINIMAX(state, depth, maximizingPlayer):
   if depth == 0 or TERMINAL-TEST(state):
      return EVAL(state)
   if maximizingPlayer:
      value = -infinity
      for each child of state:
         value = max(value, MINIMAX(child, depth-1, False))
      return value
   else:
      value = +infinity
      for each child of state:
         value = min(value, MINIMAX(child, depth-1, True))
      return value
```

- This precisely models the adversarial nature of the RTS game: the AI does not plan optimistically for the best case; it plans defensively/robustly for the worst case it could be forced into if the human opponent plays optimally against it, which is the correct modelling assumption against a skilled human competitor.
- The final move chosen by the AI is the child of the root (MAX) node with the highest backed-up minimax value — i.e., the move that maximizes the AI's guaranteed worst-case outcome.

ii. Computational Challenges of Large Game Trees

- Branching factor: RTS games typically allow a very large number of possible actions per turn (unit movement, production, ability usage, target selection, and their combinations), so the number of children per node can be enormous compared to simpler games like chess or tic-tac-toe.
- Exponential growth with depth: since the total number of nodes at depth d is roughly b^d (b = branching factor), even a modest increase in look-ahead depth causes an explosive increase in the number of states that must be evaluated — full-depth minimax quickly becomes computationally infeasible within strict real-time limits.
- Memory and time pressure: storing/evaluating a significant fraction of this exponential tree consumes substantial memory and CPU time, directly competing with the game engine's own real-time rendering, physics, and networking needs — the AI cannot simply be given unlimited computation, since the game must remain responsive to the human player.

- Evaluation cost: unlike simple games, RTS states can be expensive to evaluate accurately (many units, resources, and map features to consider), so even the leaf-evaluation step itself is not free, compounding the overall computational burden.

iii. How Alpha-Beta Pruning Improves Efficiency, with Detailed Reasoning

Alpha-Beta pruning augments minimax with two bounds tracked while traversing the tree: alpha, the best (highest) value the maximizing player can guarantee so far along the current path, and beta, the best (lowest) value the minimizing player can guarantee so far along the current path.

```
function ALPHA-BETA(state, depth, alpha, beta, maximizingPlayer):
  if depth == 0 or TERMINAL-TEST(state):
    return EVAL(state)
  if maximizingPlayer:
    value = -infinity
    for each child of state:
      value = max(value, ALPHA-BETA(child, depth-1, alpha, beta, False))
      alpha = max(alpha, value)
      if alpha >= beta: break   // beta cut-off (prune)
    return value
  else:
    value = +infinity
    for each child of state:
      value = min(value, ALPHA-BETA(child, depth-1, alpha, beta, True))
      beta = min(beta, value)
      if alpha >= beta: break   // alpha cut-off (prune)
    return value
```

- Pruning logic: whenever, while exploring a branch, the algorithm discovers that the minimizing player already has an option elsewhere that is worse for the maximizer than a value the maximizer has already secured through a different branch (alpha >= beta), the remainder of that branch cannot possibly change the final decision at the root, no matter what value it ultimately evaluates to — so it is safely skipped ("pruned") without being fully explored.
- Correctness preserved exactly: alpha-beta pruning is provably guaranteed to return the identical decision that full minimax would have returned; it only skips subtrees that are mathematically guaranteed to be irrelevant to the final answer, never sacrificing correctness for speed.
- Effect of move ordering: if the most promising moves are evaluated first at each node (e.g., via a heuristic move-ordering function, previous iteration's best move, or a transposition table), pruning becomes dramatically more effective. In the best case, alpha-beta reduces the effective branching factor from b to roughly sqrt(b), which — for the same time budget — can roughly double the achievable search depth compared to plain minimax, a critical advantage for meeting the RTS game's strict real-time constraints while still "seeing" further ahead.

iv. Designing an Evaluation Function and Justifying the Factors Considered

Because the full RTS game tree cannot realistically be searched to true terminal states within real-time limits, a depth-limited search must rely on a heuristic evaluation function EVAL(state) at the cut-off depth. A well-designed evaluation function typically combines several weighted factors:

Factor	What It Measures	Why It Matters
Material / unit balance	Number and relative strength of the AI's units vs. the opponent's	A direct proxy for current combat power and likely near-term battle outcomes
Resource control	Economy, territory, and resource nodes held by each side	Strong economy translates into greater future military and strategic strength, even if not immediately visible in unit counts
Positional advantage	Unit positioning, map control, terrain, defensive structures held	Good positioning can offset raw numerical disadvantage and predicts future tactical opportunities
Threat / opportunity assessment	Immediate tactical threats to the AI, or exploitable weaknesses in the opponent	Captures short-term tactical volatility that pure counts of units/resources may miss

- These individual factors are typically combined into a single scalar score via a weighted linear (or more complex, e.g., learned) combination: EVAL(state) = w1*Material + w2*Resources + w3*Position + w4*Threats, where the weights can be manually tuned through playtesting or learned automatically via self-play / reinforcement learning so the evaluation genuinely reflects meaningful strategic priorities rather than arbitrary designer guesses.
- A good evaluation function must also be computationally cheap to evaluate, since it will be called at every leaf of a potentially large search tree — an accurate but extremely slow evaluation function can be counter-productive if it forces a shallower search depth than a faster, slightly less accurate one.

v. Strategy for Balancing Optimality and Real-Time Performance

- Iterative deepening alpha-beta search: run alpha-beta to depth 1, then depth 2, then depth 3, and so on, always retaining the best move found at the most recently completed depth. Because the algorithm can be interrupted at any point by the strict time limit and still return a valid, reasonable move (from the last fully completed depth), this directly resolves the tension between wanting deeper search and needing a hard real-time guarantee.
- Move ordering and transposition tables: trying previously good moves first (e.g., the best move found at a shallower iterative-deepening depth, or moves stored in a transposition table from previously visited, transposed game states) substantially increases the pruning efficiency of alpha-beta, allowing more effective depth within the same time budget.
- Depth-limited search with a well-tuned evaluation function: as discussed above, trading exact terminal-state optimality for a fast, reasonably accurate heuristic evaluation is unavoidable in real-time RTS play, since true minimax optimality over the full game tree is computationally unattainable at this scale.
- Additional practical techniques: quiescence search (searching a little deeper in "volatile" tactical positions to avoid misjudging a state that is about to change sharply) and time-management heuristics (allocating more of the time budget to critical decision points and less to clearly one-sided positions) further improve the practical balance between decision quality and responsiveness.
- Overall, the combination of alpha-beta pruning with iterative deepening, strong move ordering, and a carefully weighted, cheaply computable evaluation function is the standard, well-justified approach used in real competitive game AI to balance decision quality against strict real-time constraints.